

DBMS -- SQL Commands (DDL, DML, DCL, TCL)

Target: Name all 4 language types + their commands in 20s. Explain DROP vs TRUNCATE vs DELETE clearly.

The 4 SQL Language Types -- Overview

DBMS Language

DDL | DML | DCL | TCL

Type

Full Form

Purpose

Commands

Rollback?

DDL

Data Definition Language

Define/modify DB structure (schema)

CREATE, ALTER, DROP, TRUNCATE, RENAME

Auto-committed

DML

Data Manipulation Language

Manipulate data inside tables

SELECT, INSERT, UPDATE, DELETE

[OK] Can rollback

DCL

Data Control Language

Control access/permissions

GRANT, REVOKE

Auto-committed

TCL

Transaction Control Language

Manage transactions

COMMIT, ROLLBACK, SAVEPOINT

-- (controls others)

Key distinction to remember: DDL changes the structure. DML changes the data.

DDL -- Data Definition Language

DDL commands define or modify the schema (structure) of the database.

They are auto-committed -- you cannot rollback a DDL command.

CREATE

Used to create a new database or table.

```sql

-- Create a database

CREATE DATABASE Employee;

-- Show all databases

SHOW DATABASES;

-- Create a table

CREATE TABLE students (

Roll\_No INT(3),

NAME VARCHAR(20),

SUBJECT VARCHAR(20)

);

DROP

Used to permanently delete a database or table -- including all its data and structure.

Cannot be rolled back.

```
```sql
-- Drop a table
DROP TABLE table_name;
-- Drop an entire database
DROP DATABASE student_data;
```
```

DROP removes everything -- structure + data. No going back.

TRUNCATE

Used to remove all rows from a table but keep the table structure intact.

Faster than DELETE because it doesn't log individual row deletions.

```
```sql
TRUNCATE TABLE table_name;
-- Example: remove all rows from student_details
TRUNCATE TABLE student_details;
```
```

DROP vs TRUNCATE vs DELETE -- The Most Asked Comparison

Feature

DROP

TRUNCATE

DELETE

What it removes

Table structure + all data

All rows (keeps structure)

Selected rows (or all, with WHERE)

WHERE clause

No

No

[OK] Yes

Rollback possible

No (DDL)

No (DDL)

[OK] Yes (DML)

Speed

Fast

Very fast (no row-level log)

Slow (logs each row)

Resets AUTO\_INCREMENT

[OK] Yes (table gone)

[OK] Yes

No

Triggers fired

No

No

[OK] Yes

Category

DDL

DDL

DML

Interview one-liner:

"DELETE removes specific rows and can be rolled back. TRUNCATE removes all rows fast but can't be rolled back -- it's DDL. DROP removes the entire table including structure -- also DDL, no rollback."

## ALTER

Used to modify an existing table -- add, drop, or modify columns, or add constraints.

```
```sql
-- ADD a new column
ALTER TABLE table_name
ADD (column_name datatype);
-- Example: Add AGE and COURSE columns to student table
ALTER TABLE student
ADD (AGE NUMBER(3), COURSE VARCHAR(40));
-- DROP a column
ALTER TABLE table_name
DROP COLUMN column_name;
-- MODIFY an existing column's datatype
ALTER TABLE table_name
MODIFY column_name new_datatype;
```
```

## DML -- Data Manipulation Language

DML commands read or modify the data inside tables.

They are not auto-committed -- you can rollback them using TCL commands.

## SELECT

Fetch data from one or more tables.

```
```sql
-- Fetch specific column
SELECT column_name FROM table_name;
-- Fetch all columns
SELECT * FROM table_name;
-- Real examples
SELECT EMP_NAME FROM EMPLOYEES;
SELECT * FROM EMPLOYEES;
-- With condition
SELECT * FROM EMPLOYEES WHERE EMP_AGE > 25;
```
```

## INSERT INTO

Add a new row into a table.

```
```sql
-- Method 1: Insert values for all columns (in order)
INSERT INTO table_name VALUES (value1, value2, value3);
-- Example
INSERT INTO student VALUES ('5', 'HARSH', 'WEST BENGAL', 19);
-- Method 2: Insert into specific columns only
INSERT INTO student (Roll_NO, Name, Age)
VALUES ('5', 'PRATIK', '19');
```
```

## UPDATE

Modify existing data in a table.

```
```sql
-- Basic syntax
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
-- Example 1: Set salary to 10000 for employees older than 25
UPDATE EMPLOYEES
SET EMP_SALARY = 10000
WHERE EMP_AGE > 25;
-- Example 2: Set specific employee salary
UPDATE EMPLOYEES
SET EMP_SALARY = 120000
WHERE EMP_NAME = 'APOORV';
```
```

Always use WHERE with UPDATE -- without it, every row gets updated.

## DELETE

Remove specific rows from a table.

```

```sql
-- Delete with condition
DELETE FROM table_name WHERE condition;
-- Example: delete all employees in HR dept
DELETE FROM EMPLOYEES WHERE DEPT = 'HR';
-- Delete ALL rows (can be rolled back -- unlike TRUNCATE)
DELETE FROM table_name;
```

```

## DCL -- Data Control Language

Controls who can do what in the database.

### GRANT

Give a user access/privileges to a database object.

### sql

```

GRANT SELECT, INSERT ON students TO user_name;
GRANT ALL PRIVILEGES ON database_name TO user_name;
REVOKE

```

Take back privileges previously given via GRANT.

### sql

```

REVOKE SELECT ON students FROM user_name;

```

## TCL -- Transaction Control Language

Controls the outcome of DML transactions (groups of SQL operations).

### COMMIT

Permanently save all changes made in the current transaction.

### sql

```

COMMIT;

```

-- After COMMIT, changes cannot be rolled back

### ROLLBACK

Undo all changes made since the last COMMIT or SAVEPOINT.

### sql

```

ROLLBACK;

```

-- Reverts all uncommitted DML changes

### SAVEPOINT

Set a checkpoint within a transaction. You can rollback to a specific savepoint instead of rolling back everything.

### ```sql

```

SAVEPOINT savepoint_name;

```

-- Example

```

INSERT INTO students VALUES (1, 'Avanish');

```

```

SAVEPOINT sp1;

```

```

INSERT INTO students VALUES (2, 'Rahul');

```

```

ROLLBACK TO sp1; -- undoes Rahul's insert; Avanish's insert is safe

```

```

COMMIT; -- saves Avanish permanently
```

```

TCL Transaction Flow -- Visual

```

```

```

BEGIN (transaction starts automatically with first DML)

INSERT / UPDATE / DELETE (DML operations)

SAVEPOINT sp1 (optional checkpoint)

more DML...

ROLLBACK TO sp1 (undo back to checkpoint)

OR

COMMIT (save everything permanently)

OR

ROLLBACK (undo everything since last COMMIT)

```

```

```

Quick Cheat Sheet

DDL CREATE, ALTER, DROP, TRUNCATE, RENAME changes structure auto-committed

DML SELECT, INSERT, UPDATE, DELETE changes data can rollback

DCL GRANT, REVOKE controls access auto-committed

TCL COMMIT, ROLLBACK, SAVEPOINT controls txn wraps DML

DROP = delete table + structure (no rollback)

TRUNCATE = delete all rows, keep structure (no rollback, very fast)

DELETE = delete specific rows (can rollback, fires triggers)

```

```

```

Your 40-Second Script -- SQL Languages

"SQL is divided into 4 categories. DDL -- Data Definition Language -- defines the structure: CREATE, ALTER, DROP, TRUNCATE. It's auto-committed so you can't rollback DDL. DML -- Data Manipulation Language -- handles data: SELECT, INSERT, UPDATE, DELETE. DML can be rolled back. DCL controls permissions -- GRANT and REVOKE. TCL manages transactions -- COMMIT saves permanently, ROLLBACK undoes changes, SAVEPOINT sets a checkpoint mid-transaction."

Follow-Up Questions (Expect These)

Q: Can you rollback a DROP statement?

No. DROP is DDL and is auto-committed. Once dropped, the table is gone unless you have a backup.

Q: What happens if you UPDATE without a WHERE clause?

Every row in the table gets updated. Always double-check your WHERE clause before running UPDATE or DELETE.

Q: What is the difference between GRANT and REVOKE?

GRANT gives privileges to a user. REVOKE takes them back. Both are DCL commands and auto-committed.

Q: Can you ROLLBACK after a COMMIT?

No. COMMIT makes the changes permanent and durable. You cannot undo it with ROLLBACK -- only a new DML operation can reverse it.